```python
#Determine user input
something = input("Enter something : ")
print("You entered : ", something)
print("Data type of num : ", type(something))
```

```
Enter something : fhsjigvhuhb
You entered :  fhsjigvhuhb
Data type of num :  <class 'str'>
```

```python
num = int(input("Enter a number : "))
print("Square : ",(num ** 2))
print("Cube : ",(num ** 3))
print("Square root : ",(num ** 0.5))
```

```
Enter a number : 4
Square :  16
Cube :  64
Square root :  2.0
```

```python
#arithmentic operation
a=int(input("Enter first no : "))
b=int(input("Enter second no : "))
print("addition is : ",(a+b))
print("substraction is : ",(a-b))
print("multiplication is : ",(a*b))
print("division is :",(a/b))
print("modulus is : ",(a%b))
print("floor division is : ",(a//b))
print("exponent is : ",(a**b))
```

```
Enter first no : 1
Enter second no : 2
addition is :  3
substraction is :  -1
multiplication is :  2
division is : 0.5
modulus is :  1
floor division is :  0
exponent is :  1
```

```python
#comparison operator
a=int(input("Enter first no : "))
b=int(input("Enter second no : "))
print((a>b))#greater then
print((a<b))#less then
print((a==b))#equal to
print((a!=b))#not equal to
print((a>=b))#greater then or equal to
print((a<=b))#less then or equal to
```

```
Enter first no : 2
Enter second no : 3
False
True
False
True
False
True
```

```
#area of right angled triangle
l=float(input("Enter length : "))
h=float(input("Enter breadth : "))
print("Area of the right angled triange : ",(0.5*l*b))
```

```
Enter length : 2
Enter breadth : 3
Area of the right angled triange :  3.0
```

```
# average
a=float(input("Enter first no : "))
b=float(input("Enter second no : "))
c=float(input("Enter third no : "))
avg = (a+b+c)/3
print("Average : ",avg)
```

```
Enter first no : 1
Enter second no : 2
Enter third no : 3
Average :  2.0
```

```
#simple interest
p=float(input("Enter principal value : "))
r=float(input("Enter interest rate : "))
n=float(input("Enter no of year: "))
si = (p*r*n)/100
print("Simple Interest : ",si)
```

```
Enter principal value : 1000
Enter interest rate : 5
Enter no of year: 3
Simple Interest :  150.0
```

```
# celsius to fahrenheit
c = float(input("Enter the temperature in celsius : "))
f = ((9/5)*c)+32
print("Temperature in Fahrenheit : ",f)
```

```
Enter the temperature in celsius : 29
Temperature in Fahrenheit :  84.2
```

```python
# euclidean distance
x1 = float(input("Enter first x co-ordinate : "))
y1 = float(input("Enter first y co-ordinate : "))
x2 = float(input("Enter second x co-ordinate : "))
y2 = float(input("Enter second y co-ordinate : "))
dist = ((((x2-x1)**2)+((y2-y1)**2)) ** 0.5)
print("Euclidean distance : ",dist)
```

```
Enter first x co-ordinate : 1
Enter first y co-ordinate : 2
Enter second x co-ordinate : 3
Enter second y co-ordinate : 4
Euclidean distance :  2.8284271247461903
```

```python
# swap 2 variables using comma,temp,bitwiseXOR
a=3
b=4
a,b=b,a
print(a,b)
```

```
4 3
```

```python
a=3
b=4
t=a
a=b
b=t
print(a,b)
```

```
4 3
```

```python
a=9
b=4
a= a^b
b = a^b
a = a^b
print(a,b)
```

```
4 9
```

```python
# solution to quadratic equation
a=float(input("Enter first co-effiecient : "))
b=float(input("Enter second co-effiecient : "))
c=float(input("Enter third co-effiecient : "))
D = ((b**2)-(4*a*c))
if D > 0:
    x1=(((b*(-1))+(D ** 0.5))/(2*a))
    x2=(((b*(-1))-(D ** 0.5))/(2*a))
    print("Roots of the equation are : ",x1,"and",x2)
elif D==0:
    x=((b*(-1))/(2*a))
```

```
    print("Root of the equation is : ",x)
else:
    print("Real solution doesn't exist")
    x1=(((b*(-1))+(D ** 0.5))/(2*a))
    x2=(((b*(-1))-(D ** 0.5))/(2*a))
    print("Complex Roots of the equation are : ",x1,"and",x2)
```

```
Enter first co-effiecient : 1
Enter second co-effiecient : 1
Enter third co-effiecient : 1
Real solution doesn't exist
Complex Roots of the equation are :  (-
0.49999999999999994+0.8660254037844386j) and (-0.5-
0.8660254037844386j)
```

```python
# accept 3 sides of triangle and tell its type
a = float(input("Enter 1st Side of the triangle : "))
b = float(input("Enter 2nd Side of the triangle : "))
c = float(input("Enter 3rd Side of the triangle : "))

if(a==b and b==c):
    print("Equilateral Triangle")
elif(a==b or b==c or a==c):
    print("Isoceles Triangle")
else:
    print("Scalene Triangle")
```

```
Enter 1st Side of the triangle : 1
Enter 2nd Side of the triangle : 1
Enter 3rd Side of the triangle : 1
Equilateral Triangle
```

```python
#function solver

print("f(x) = ax^3 - bx^2 + cx -d            if x > k")
print("f(x) = 0                             if x = k")
print("f(x) = -ax^3 + bx^2 - cx +d          if x < k")

a=float(input("Enter value of a : "))
b=float(input("Enter value of b : "))
c=float(input("Enter value of c : "))
d=float(input("Enter value of d : "))
k=float(input("Enter value of k : "))
x=float(input("Enter value of x : "))

if(x>k):
    f = (a*(x**3))-(b*(x**2))+(c*x)-d
    print(f"Solution : {f}")
elif(x==k):
    print("Solution : 0")
else:
    f = (a*(x**3))-(b*(x**2))+(c*x)-d
    print(f"Solution : {f}")
```

```
f(x) = ax^3 - bx^2 + cx -d            if x > k
f(x) = 0                              if x = k
f(x) = -ax^3 + bx^2 - cx +d           if x < k
Enter value of a : 1
Enter value of b : 2
Enter value of c : 3
Enter value of d : 4
Enter value of k : 5
Enter value of x : 6
Solution : 158.0
```

```python
# BMI calculator
w = float(input("Enter weight in kg : "))
h = float(input("Enter height in m : "))

bmi = w/(h**2)

if(bmi <= 18.4):
    print(f"BMI : {bmi:.2f} Status : Underweight")
elif(bmi <= 24.9):
    print(f"BMI : {bmi:.2f} Status : Normal")
elif(bmi <= 39.9):
    print(f"BMI : {bmi:.2f} Status : Overweight")
else:
    print(f"BMI : {bmi:.2f} Status : Obese")
```

```
Enter weight in kg : 105
Enter height in m : 1.83
BMI : 31.35 Status : Overweight
```

```python
#DISTANCE CONVERTER

print("Enter the value of the convereter u want to use")
print("1. Inches to Feet ")
print("2. Centimeter to Meter ")

choice = int(input("Enter the value : "))

if (choice == 1):
    inch = float(input("Enter distance in inches : "))
    feet = inch/12
    print(f"feet : {feet:.2f}")
elif(choice == 2):
    cm = float(input("Enter distance in centimeter : "))
    m = cm/100
    print(f"meter : {m:.2f}m")
else:
    print("Invalid Choice")
```

```
Enter the value of the convereter u want to use
1. Inches to Feet
2. Centimeter to Meter
Enter the value : 2
Enter distance in centimeter : 183
meter : 1.83m
```

```python
#read and display string
s = input("Enter String : ")
print(s)
```

```
Enter String : Hello World1
Hello World1
```

```python
# length of string
s = "Hello World"
print(len(s))
```

```
11
```

```python
# lower case
s = "Python Programming"
s=s.lower()
print(s)
```

```
python programming
```

```python
# concat
s1="Data"
s2="Science"
s = s1 + " " + s2
print(s)
```

```
Data Science
```

```python
#remove trailing spaces
s = "    Data Analysis    "
print(s)
s = s.strip()
print(s)
```

```
    data analysis
data analysis
```

```python
#odd string elements
s ="Data Science"
print(s[1::2])
```

```
aaSine
```

```python
#check if palindrom or not
s = input("Enter String : ")
rs = s[::-1]
if (s == rs ):
    print("Palindrome")
else :
    print("Not Palindrome")
```

```
Enter String : amma
Palindrome
```

```python
#string reverse
s="Hello"
print(s[::-1])
```

```
olleH
```

```python
# the seperator and joiner
s = "Hello world! I am Tired saying this same word"
a = s.split()
sym = input("enter joiner : ")
print(s)
print(a)
print(sym.join(a))
```

```
enter joiner : -
Hello world! I am Tired saying this same word
['Hello', 'world!', 'I', 'am', 'Tired', 'saying', 'this', 'same',
'word']
Hello-world!-I-am-Tired-saying-this-same-word
```

```python
# input words then reverse words order then join by hypen
s = input("Enter the words : ")
a = s.split()
a = a[::-1]
a = '-'.join(a)
print(a)
```

```
Enter the words : Hello world! I am Tired saying this same word
word-same-this-saying-Tired-am-I-world!-Hello
```

```python
#sum of 1st n natural numbers
sum = 0
n=int(input("Enter no of elements : "))
for i in range(1,n+1):
    sum+=i
print(sum)
```

```
Enter no of elements : 10
55
```

```python
#count number of even and odd numbers in the given series
p=int(input("enter p :"))
q=int(input("enter q :"))
even = 0
odd = 0
for i in range(p,q+1):
    if(i%2 == 0):
        even+=1
    else:
        odd+=1

print(f"Total Even numbers : {even}")
print(f"Total Odd numbers : {odd}")
```

```
enter p :51
enter q :60
Total Even numbers : 5
Total Odd numbers : 5
```

```python
#prime checker
n = int(input("Enter a number : "))
i = 2
count = 0

while (i < n):
    if (n % i == 0):
        count += 1
        break
    i += 1

if (count == 0 and n > 1):
    print("Prime")
else:
    print("Not prime")
```

```
Enter no : 4
```

```python
#fibonnaci
n = int(input("Enter the last term u want in fibonacci series : "))
a=0
b=1
```

```
while(a <= n):
    print(a,end=" ")
    a,b=b,a+b
```

```
Enter the last term u want in fibonacci series : 13
0 1 1 2 3 5 8 13
```

```
#nested pattern loop
for i in range(1,6):
    for j in range(6-i):
        print(" ",end="")
    for k in range(2*i-1):
        print("*",end="")
    print()
```

```
     *
    ***
   *****
  *******
 *********
```

```
#pattern 1
for i in range(1,6):
    for j in range (6-i):
        print("*",end=" ")
    print()
```

```
* * * * *
* * * *
* * *
* *
*
```

```
#pattern 2
for i in range(1,6):
    for j in range(1,i+1):
        print(j,end="")
    print()
```

```
1
12
123
1234
12345
```

```
#pattern 3
n1 = 3
for i in range(n1):
    for j in range(i+1):
        print("* ",end="")
    print()
```

```python
for i in range(n1):
    for j in range(i,n1-1):
        print("* ",end="")
    print()
```

```
*
* *
* * *
* *
*
```

```python
#fizzbuzz
for i in range(1,51):
    if(i%3==0 and i%5!=0):
        print("Fizz")
    elif(i%3!=0 and i%5==0):
        print("Buzz")
    elif(i%3==0 and i%5==0):
        print("FizzBuzz")
    else:
        print(i)
```

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
Fizz
22
23
Fizz
Buzz
26
```

```
Fizz
28
29
FizzBuzz
31
32
Fizz
34
Buzz
Fizz
37
38
Fizz
Buzz
41
Fizz
43
44
FizzBuzz
46
47
Fizz
49
Buzz
```

```
#string operations
s = input("enter string sentence: ")
print(s.upper())
print(s.lower())
print(s.capitalize())
s=s.split()
print(s)
s='_@_'.join(s)
print(s)
```

```
enter string sentence: Hello world i am python not language snake
HELLO WORLD I AM PYTHON NOT LANGUAGE SNAKE
hello world i am python not language snake
Hello world i am python not language snake
['Hello', 'world', 'i', 'am', 'python', 'not', 'language', 'snake']
Hello_@_world_@_i_@_am_@_python_@_not_@_language_@_snake
```

```
#continue
for i in range(10):
    if(i%2==0):
        continue
    print(i)
```

```
1
3
```

```
5
7
9

#pass and else
for i in range(11):
    if(i==3):
        pass
    print(i)
else:
    print("Loop complete")

0
1
2
3
4
5
6
7
8
9
10
Loop complete
```

Create a list of the first 10 positive integers and print the sum of the even numbers in the list

```
num=list(range(1,11))

list

sum=0
for i in num:
    if(i%2==0):
        sum+=i
print(sum)

30
```

Demonstrate the following operations on a number list using list functions a.insert 13 at position 4 b.Sort the list in ascending order c.delete the last element d.remove 31 e.Reverse the list f.Append one number to list g.extend the list with [20,30,40] h.print the number of elements in the list using while

```
num=list(range(1,11))
print(num)

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

num.insert(4,13)
print(num)

[1, 2, 3, 4, 13, 5, 6, 7, 8, 9, 10]

num.sort()
print(num)

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 13]

num.pop()
print(num)

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

num.remove(31)

---------------------------------------------------------------------------
-----
ValueError                                Traceback (most recent call
last)
<ipython-input-25-04da6c47085e> in <module>
----> 1 num.remove(31)

ValueError: list.remove(x): x not in list
```

```
num.sort(reverse=True)
print(num)
```

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

```
num.append(45)
print(num)
```

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 45]
```

```
l=[20,30,40]
num.extend(l)
print(num)
```

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 45, 20, 30, 40]
```

```
i=0
while i < len(num):
    print(num[i])
    i+=1
```

```
10
9
8
7
6
5
4
3
2
1
45
20
30
40
```

Given the tuple (10, 20, 30, 40, 50), use slicing to print a new tuple containing only the last three elements.

```
num=(10,20,30,40,50)
print(num[2:])
```

```
(30, 40, 50)
```

Create a set of the numbers {1, 2, 3, 4, 5} and compute the intersection with {4, 5, 6, 7}; print the result.

```
set1={1,2,3,4,5}
set2={4,5,6,7}
```

```
print(set1&set2)
```

```
{4, 5}
```

Define a dictionary with the keys 'a', 'b', and 'c' and their respective values 1, 2, and 3, then update the value for key 'b' to 10 and print the updated dictionary.

```
thisdict ={
    'a' : 1,
    'b' : 2,
    'c' : 3
}
print(thisdict)
dict['b']=10
print(thisdict)
```

```
{'a': 1, 'b': 2, 'c': 3}
{'a': 1, 'b': 2, 'c': 3}
```

Using a list comprehension, create a list of squares for the numbers from 1 to 5 and print the resulting list.

```
sq = [x**2 for x in range(1,6)]
print(sq)
```

```
[1, 4, 9, 16, 25]
```

Create a tuple with elements 'cat', 'dog', and 'bird', then check and print whether 'dog' is in the tuple.

```
animal=("cat","dog","bird")
print("dog" in animal)
```

```
True
```

Given the list [1,1,2,3,5,8,13], convert it to a set and print the set's length.

```
ls = [1,1,2,3,5,8,13]
print(ls)
s = set(ls)
print(s)
print(len(s))
```

```
[1, 1, 2, 3, 5, 8, 13]
{1, 2, 3, 5, 8, 13}
6
```

Create two dictionaries {'A': [1, 2], 'B': [3, 4]} and {'B': [5, 6], 'C': [7, 8]}, then merge them into one dictionary and print the result.

```
dict1 = {'A':[1,2], 'B':[3,4]}
dict2 = {'B':[5,6], 'C':[7,8]}
merged_dict = {**dict1, **dict2}
print(merged_dict)
```

```
{'A': [1, 2], 'B': [5, 6], 'C': [7, 8]}
```

For the list [3, 5, 7, 3, 5, 9, 3], create a dictionary to count the occurrences of each element and print the dictionary.

```
elements = [3, 5, 7, 3, 5, 9, 3]
counts = {}

for num in elements:
    if num in counts:
        counts[num] += 1
    else:
        counts[num]=1
print(counts)
```

```
{3: 3, 5: 2, 7: 1, 9: 1}
```

```python
import numpy as np

#sum of element of 3 x 3
arr3d = np.random.randint(1,11,size=(3,3))
print(arr3d)
Sum = np.sum(arr3d)
print("Sum of all Elements : ",Sum)
```

```
[[ 2  5  2]
 [ 4  9  4]
 [ 2  6 10]]
Sum of all Elements :  44
```

```python
#avg of element of 4 x 4
arr4d = np.random.rand(4,4)
print(arr4d)
avg = np.mean(arr4d)
print("Average of all Elements : ",avg)
```

```
[[0.31008133 0.30666187 0.03657576 0.03351542]
 [0.40332796 0.04365721 0.43461072 0.13389531]
 [0.86378715 0.21645602 0.27535305 0.07617275]
 [0.8475716  0.15397093 0.68304676 0.56588243]]
Average of all Elements :  0.33653539215167105
```

```python
#sum of element of 2 x 3
arr2x3 = np.random.rand(2,3)
print(arr2x3)
Sum = np.sum(arr2x3[:,1])
print("Sum of all Elements of 2nd row : ",Sum)
```

```
[[0.62572517 0.34119676 0.20156804]
 [0.80421055 0.28441033 0.87016062]]
Sum of all Elements of 2nd row :  0.6256070889323414
```

```python
# find highest element in 5x5
arr5x5 = np.random.randint(1,10001,size=(5,5))
print(arr5x5)
Max = np.max(arr5x5)
print("Max among all Elements : ",Max)
```

```
[[1508 4004 1511 6881 7283]
 [ 530 7680  213 7470 5627]
 [1296 2579 2639 3159 1268]
 [1358 4418 7124  343 2146]
 [2876 8985  344  278 2659]]
Max among all Elements :  8985
```

```python
# find trace in 4x4
arr4x4 = np.random.rand(4,4)
print(arr4x4)
```

```
tr = np.trace(arr4x4)
print("sum of all diagonal elements : ",tr)
```

```
[[0.38969828 0.32538589 0.29082692 0.38099127]
 [0.87866076 0.99828621 0.93642776 0.59968742]
 [0.3412576  0.03916221 0.23970246 0.04576408]
 [0.48057372 0.27434913 0.94587712 0.26478667]]
sum of all diagonal elements :  1.8924736319021438
```

```
# find highest element in 3x3
arr3x3 = np.random.rand(3,3)
print(arr3x3)
Min = np.min(arr3x3)
print("Minimum among all Elements : ",Min)
```

```
[[0.38374901 0.23625982 0.61691999]
 [0.14352691 0.03455422 0.05900485]
 [0.64946168 0.99616589 0.63250666]]
Minimum among all Elements :  0.03455422138235509
```

```
# sum all elements > 10 in 3X3
arr3x3 = np.random.randint(1,21,size=(3,3))
print(arr3x3)
Sum = np.sum(arr3x3[arr3x3 > 10])
print("Sum of all Elements of 2nd row : ",Sum)
```

```
[[17  5  1]
 [13  1  4]
 [10  6 20]]
Sum of all Elements of 2nd row :  50
```

```
# Generate a 5x5 matrix of random numbers and find the mean of the
maximum values of each row.
arr5x5 = np.random.randint(1, 101, size=(5, 5))
print(arr5x5)
row_maxima = np.max(arr5x5, axis=1)
print("Row Max :", row_maxima)
mean_of_maxima = np.mean(row_maxima)
print("Mean of Max :", mean_of_maxima)
```

```
[[37 73 95 60 10]
 [21 89 84 70 62]
 [73 34 42 12 89]
 [24 71  6 20 25]
 [ 2 63 97 58 16]]
Row Maxima: [95 89 89 71 97]
Mean of Maxima: 88.2
```

```
# Create a 3x3 matrix and calculate the difference between the sum of
the elements in the first row and
# the sum of the elements in the last row.
```

```
arr3x3 = np.random.randint(1, 21, size=(3, 3))
print(arr3x3)
sum_first_row = np.sum(arr3x3[0, :])
sum_last_row = np.sum(arr3x3[-1, :])
difference = sum_first_row - sum_last_row
print("Difference between sum of first and last row : ", difference)
```

```
[[ 9 19 15]
 [18  5  5]
 [14  9 16]]
Difference between sum of first and last row :  4
```

```
# Generate a 6x6 matrix of random integers and replace all values
below the median with zero.
arr6x6 = np.random.randint(1, 101, size=(6, 6))
print(arr6x6)
median_val = np.median(arr6x6)
print("Median value : ",median_val)
arr6x6[arr6x6 < median_val] = 0
print("Matrix after replacing values below median with zero:\n",
arr6x6)
```

```
[[18 53  2 18 50 81]
 [12 16 15 92 29 77]
 [58 48 29 36 57 80]
 [33 69 16 18 48  9]
 [16 51 16 96 70 96]
 [56 25 44 28  7 47]]
Median value :  40.0
Matrix after replacing values below median with zero:
 [[ 0 53  0  0 50 81]
 [ 0  0  0 92  0 77]
 [58 48  0  0 57 80]
 [ 0 69  0  0 48  0]
 [ 0 51  0 96 70 96]
 [56  0 44  0  0 47]]
```

```
# Create a 4x4 matrix and compute the sum of elements in the upper
triangular part of the matrix.
arr4x4 = np.random.randint(1, 101, size=(4, 4))
print(arr4x4)
upper_tri_elements = arr4x4[np.triu_indices(4)]
sum_upper_tri = np.sum(upper_tri_elements)
print("Sum of elements in the upper triangular part : ",
sum_upper_tri)
```

```
[[90 84 62 47]
 [ 8 97 73 54]
 [97 85 61 23]
```

```
 [33 95 90 53]]
Sum of elements in the upper triangular part :  644

# Generate a 3x4 matrix of random values and find the standard
deviation of the elements in the third col.
arr3x4 = np.random.randint(1, 101, size=(3, 4))
print(arr3x4)
third_column_std = np.std(arr3x4[:, 2])
print("Standard deviation : ", third_column_std)

[[19 65 16 88]
 [12 74 19 34]
 [69 18 89 82]]
Standard deviation :  33.7276675083759
```